

SoundWave – Project Roadmap

| Spotify Clone · React + .NET 8 · Modular Monolith · CQRS + Vertical Slicing

How to Read This Roadmap

Each phase is broken into **sub-phases** tagged with the module they touch.
Finish one sub-phase fully (working code, passing tests) before moving to the next.
Do not start Phase 1 until Phase 0 is complete — the study phase will save you weeks of confusion later.

Module tags used throughout: [Identity] [Catalog] [Streaming] [Playlist] [Social]
[Analytics] [Infra] [Frontend]

Phase 0 — Study & Planning

- Goal:** Understand every technology and pattern before writing a single line of production code.
- Duration estimate:** 2–3 weeks
- Output:** You can explain every decision below without looking at notes.

0.1 — Architecture Patterns

Topic	What to study	Why you need it
Modular Monolith	How modules communicate, what a module boundary means, why no direct cross-module calls	Your entire backend is structured this way
Vertical Slice Architecture	Feature folders instead of layer folders, one folder per use case	How you will organise every .NET feature
CQRS + MediatR	Commands vs queries, IRequest<T> , pipeline behaviours	Every handler in your app follows this
Result<T> pattern	Returning errors as values, never throwing business exceptions	Replaces try/catch in all handlers
Repository pattern	Why you abstract EF Core behind an interface	Testability — you mock repos in unit tests

Topic	What to study	Why you need it
Outbox pattern	Why you write to DB before publishing to RabbitMQ	Prevents lost events on crash
Soft delete with EF Core	Global query filter <code>HasQueryFilter(e => !e.IsDeleted)</code>	Every entity uses this

Resources to look up:

- Jimmy Bogard — Vertical Slice Architecture (blog + talks)
- Milan Jovanović — Modular Monolith in .NET (YouTube)
- Nick Chapsas — MediatR pipeline behaviours (YouTube)

0.2 — RabbitMQ (most important to study before building)

Topic	What to understand
Exchange types	Direct, Topic, Fanout — you use Topic exchange
Routing keys	Pattern <code>catalog.track.uploaded</code> , how consumers bind to patterns
Queues vs exchanges	Exchange receives, queue holds, consumer reads
Dead letter queues	What happens when a consumer fails — message goes to <code>.dlq</code>
Message acknowledgement	<code>BasicAck</code> vs <code>BasicNack</code> — when to ack, when to nack
Raw	<code>RabbitMQ.Client</code> , <code>IConnection</code> , <code>IModel</code> , <code>BasicPublish</code> , <code>BasicConsume</code> — no MassTransit
Connection pooling	One <code>IConnection</code> per app, one <code>IModel</code> per operation

Key decision already made: You use raw `RabbitMQ.Client` — no MassTransit. This means more boilerplate but full control. Study the raw client specifically.

Practice exercise before Phase 2:

Build a tiny throwaway console app — Publisher sends a message to a topic exchange, Consumer reads it and prints it. Get dead-letter working. Takes 2–3 hours, saves days later.

0.3 — EF Core + MSSQL with Modular Monolith

Topic	What to understand
One DbContext per module	<code>CatalogDbContext</code> , <code>IdentityDbContext</code> etc — not one giant context
Schema-per-module in EF Core	<code>modelBuilder.HasDefaultSchema("Catalog")</code>
<code>AsNoTracking()</code> on queries	All read/query handlers use this — never loads change tracker
Fluent API configuration	<code>IEntityTypeConfiguration<T></code> per entity — no data annotations
Migrations per module	Separate migration project per module or one shared migrations project
GUID v7 as IDs	<code>Guid.CreateVersion7()</code> in .NET 9 or <code>UuidNext</code> package in .NET 8 — set <code>ValueGeneratedNever()</code> in EF config
Cross-module IDs	Plain <code>Guid</code> property, no navigation property, no <code>HasForeignKey()</code>

0.4 — HLS + FFmpeg

Topic	What to understand
What HLS is	<code>.m3u8</code> playlist + <code>.ts</code> segments — how a browser streams it
FFmpeg command for HLS	<code>ffmpeg -i input.mp3 -codec:a aac -hls_time 10 -hls_list_size 0 output.m3u8</code>
30s preview	<code>-t 30</code> flag to cut the preview playlist
<code>System.Diagnostics.Process</code>	How .NET invokes FFmpeg CLI — capture exit code + stderr
<code>hls.js</code>	How the React frontend loads and plays <code>.m3u8</code> files

Practice exercise: Run FFmpeg manually on an MP3 and produce HLS segments. Serve them with a simple static server and play them in the browser with `hls.js`. Understand the flow before building the consumer.

0.5 — Redis Patterns You Will Use

Pattern	Redis key	Used for
Play count buffer	<code>play_count:{trackId}</code>	INCR on play, Worker flushes to DB every 5 min
Playback position	<code>playback_pos:{userId}:{trackId}</code>	Resume across devices, TTL 30 days
Queue session	<code>queue:{userId}</code>	Active queue JSON, synced to DB periodically
Access token blacklist	<code>blacklist:{jti}</code>	Revoked JWTs, TTL = remaining token lifetime
Email verify token	<code>email_verify:{userId}</code>	Single-use, TTL 24h
Password reset token	<code>pwd_reset:{userId}</code>	Single-use, TTL 1h
Login fail counter	<code>login_fails:{email}</code>	INCR, TTL 15 min, lock after 5
Rate limiter	<code>ratelimit:{ip}</code>	Sliding window per IP

0.6 — File Storage Decision

For development: Local file system via `IFileStorage` abstraction.

```
wwwroot/  
  uploads/  
    raw/{trackId}.mp3      <- uploaded file lives here  
    hls/{trackId}/  
      playlist.m3u8        <- FFmpeg output  
      preview.m3u8  
      segment_000.ts  
      segment_001.ts
```

ASP.NET Core `UseStaticFiles()` serves the `wwwroot/` folder directly.

`.gitignore` must include `wwwroot/uploads/`.

For production (post-launch): Swap `LocalFileStorage` for `AzureBlobStorage` or `S3FileStorage` — zero Application layer changes because of `IFileStorage` abstraction.

Recommendation: Start with local file system. Do not add cloud storage complexity until the core pipeline works end-to-end.

0.7 — Docker Compose Local Stack

Before writing any app code, get this running locally:

```
mssql      → port 1433
redis      → port 6379
rabbitmq   → port 5672 (management UI: 15672)
seq        → port 5341 (UI: 80)
```

Add ElasticSearch and Qdrant only when you reach Phase 5.

0.8 — What You Might Have Missed (Study These Too)

Topic	Why
<code>IOptions<T></code> pattern in .NET	How to inject config (connection strings, JWT settings, RabbitMQ config) cleanly
<code>ProblemDetails</code> RFC 7807	Your global exception middleware returns this format
<code>IHostedService</code> / <code>BackgroundService</code>	How .NET Worker Services work — <code>OutboxProcessor</code> , play count flusher are these
FluentValidation pipeline behaviour	<code>IPipelineBehavior<TRequest, TResponse></code> — runs before every command handler
EF Core global query filters	<code>HasQueryFilter</code> for soft delete — how to bypass it for admin queries
Correlation IDs in ASP.NET Core	Middleware that generates/reads <code>X-Correlation-ID</code> header and passes it to Serilog
JWT claims and <code>[Authorize(Roles=)]</code>	How role-based auth works in ASP.NET Core — Listener vs Artist vs Admin

Phase 1 — Foundation & Core Modules

Goal: Running API with auth, catalog browsing, and basic playlist management. No streaming yet.

Technologies: .NET 8, MSSQL, EF Core, MediatR, FluentValidation, Serilog, Redis (auth only), JWT

File storage: Not needed yet

Frontend: Basic React shell with routing only

1.1 — Project Skeleton [Infra]

- Create solution with module project structure
 - One [DbContext] per module, each with its own MSSQL schema
 - Global exception middleware returning [ProblemDetails]
 - Serilog configured with file sink + Seq sink
 - Correlation ID middleware
 - Health check endpoints [/health] and [/health/ready]
 - Docker Compose with MSSQL + Redis + Seq running
 - EF Core migrations set up
 - [BaseEntity] with [CreatedBy], [CreatedDate], [UpdatedBy], [UpdatedDate], [IsDeleted]
 - Global soft-delete query filter on all entities
 - [Result<T>] and [Error] types defined
 - [Guid.CreateVersion7()] helper or [UuidNext] package
-

1.2 — Identity Module: Registration & Login [Identity]

Features: Register, Email verification, Login, Logout

Tables: [Identity.Users], [Identity.RefreshTokens]

Redis: [email_verify:{userId}], [login_fails:{email}], [blacklist:{jti}]

- [RegisterCommand] → BCrypt hash → insert [Identity.Users] → write [OutboxMessage] (UserRegistered)
- [LoginCommand] → BCrypt verify → issue JWT (15 min) + refresh token (7 days, hashed in [RefreshTokens])
- [RefreshTokenCommand] → validate hash → rotate token → issue new JWT

- `LogoutCommand` → revoke refresh token in DB → add jti to Redis blacklist
 - `VerifyEmailCommand` → validate Redis key → set `IsEmailVerified = 1`
 - Account logout: Redis `login_fails:{email}` INCR, lock after 5
 - `[Authorize]` middleware wired up with role claims
 - FluentValidation on all commands
 - xUnit tests for all handlers
-

1.3 — Identity Module: Password Reset `[Identity]`

Features: Forgot password, Reset password

Redis: `pwd_reset:{userId}` TTL 1h

- `ForgotPasswordCommand` → generate token → store in Redis → log email for now
 - `ResetPasswordCommand` → validate Redis token → BCrypt new hash → delete Redis key
 - xUnit tests
-

1.4 — Catalog Module: Genres & Artists `[Catalog]`

Features: Admin creates genres/moods, Artist profile browsing

Tables: `Catalog.Genres`, `Catalog.Artists`

- `CreateGenreCommand` `[Admin]` → insert `Catalog.Genres`
 - `GetGenresQuery` → list all genres/moods (Redis cached)
 - `ApplyForArtistCommand` `[Listener]` → insert `Catalog.Artists` with `IsApproved = false`
 - `ApproveArtistCommand` `[Admin]` → set `IsApproved = true` → write `OutboxMessage` (ArtistApproved) → write `Infra.AuditLogs`
 - `GetArtistProfileQuery` → returns artist + top tracks + albums
 - xUnit tests
-

1.5 — Catalog Module: Albums & Tracks `[Catalog]`

Features: Artist creates albums, adds track metadata (no audio yet)

Tables: `Catalog.Albums`, `Catalog.Tracks`, `Catalog.TrackGenres`, `Catalog.AlbumGenres`,

`Catalog.AlbumArtists`, `Catalog.TrackArtists`

- `CreateAlbumCommand` `[Artist]` → insert `Catalog.Albums`
 - `AddTrackToAlbumCommand` `[Artist]` → insert `Catalog.Tracks` (Status = Pending, no file yet)
 - `EditTrackMetadataCommand` `[Artist]` → update title, genres, featured artists
 - `PublishAlbumCommand` `[Artist]` → set `IsPublished = true` (only if all tracks Ready)
 - `GetAlbumQuery` → album + ordered track listing
 - `GetNewReleasesQuery` → albums sorted by `ReleaseDate DESC`
 - xUnit tests
-

1.6 — Playlist Module: Core Playlists `[Playlist]`

Features: Create/edit/delete playlists, add/remove/reorder tracks, Liked Songs

Tables: `Playlist.Playlists`, `Playlist.PlaylistTracks`, `Playlist.LikedTracks`, `Playlist.LikedAlbums`, `Playlist.LikedPlaylists`

- `CreatePlaylistCommand` `[Listener]`
 - `EditPlaylistCommand` `[Listener]` → rename, change visibility, cover art
 - `DeletePlaylistCommand` `[Listener]` → soft delete, 403 if `IsSystem = true`
 - `AddTrackToPlaylistCommand` → insert `PlaylistTracks`, update denormalized counts
 - `RemoveTrackFromPlaylistCommand` → delete row, re-gap positions, update counts
 - `ReorderPlaylistTracksCommand` → bulk update positions
 - `LikeTrackCommand` → insert `LikedTracks` + add to Liked Songs `PlaylistTracks`
 - `UnlikeTrackCommand` → delete from both
 - `LikeAlbumCommand` / `UnlikeAlbumCommand`
 - `GetLibraryQuery` → user's playlists, liked albums, followed artists
 - xUnit tests
-

1.7 — Infra Module: Outbox Processor `[Infra]`

Features: Background Worker publishes `OutboxMessages` to RabbitMQ

Tables: `Infra.OutboxMessages`

- `OutboxProcessorWorker` — `BackgroundService` polls `OutboxMessages` WHERE `ProcessedAt IS NULL`
 - Publishes to RabbitMQ topic exchange `soundwave.events`
 - Sets `ProcessedAt` on success, increments `RetryCount` on failure
 - Dead after `RetryCount = 3`
 - Serilog logs every publish with correlation ID
 - Note: RabbitMQ consumers not built yet — messages published and ignored for now
-

1.8 — Frontend Shell `[Frontend]`

Technologies: React + Vite, React Router v6, TanStack Query, Axios, Tailwind CSS, React Context + useReducer

- Project scaffold with Vite
 - Routing: `/`, `/login`, `/register`, `/artist/:id`, `/album/:id`, `/playlist/:id`, `/library`
 - Axios instance with JWT interceptor (attach token, refresh on 401)
 - Auth context (login state, user role)
 - Player context shell (useReducer — no audio yet, just state structure)
 - Basic pages — working navigation, no styling yet
-

Phase 2 — Streaming Pipeline

Goal: Upload audio, process via FFmpeg, stream HLS, record play events.

New technologies: RabbitMQ consumers, FFmpeg CLI via `System.Diagnostics.Process`, `hls.js`

File storage: Local disk (`IFileStorage` abstraction, `LocalFileStorage` implementation)

2.1 — File Storage Abstraction `[Infra]`

- Define `IFileStorage` in Domain layer: `SaveAsync`, `ReadAsync`, `DeleteAsync`, `GetUrl`
 - Implement `LocalFileStorage` in Infrastructure
 - Serve `wwwroot/uploads/` via `UseStaticFiles()` in dev
 - Unit test `LocalFileStorage`
-

2.2 — Track Upload `[Catalog]`

Features: Artist uploads raw audio file

Tables: `Catalog.Tracks` (Status=Pending), `Catalog.TrackFiles`, `Infra.OutboxMessages`

- `UploadTrackCommand` `[Artist]` → validate MIME + magic bytes → stream to `IFileStorage` raw/ → insert `TrackFiles` (Status=Pending) → write `OutboxMessage` (TrackUploaded) — all in one EF transaction
 - `GetTrackStatusQuery` → return `TrackFiles.Status` + `FailureReason`
 - FluentValidation: max 50MB, allowed formats (mp3/flac/aac/wav) only
 - xUnit tests
-

2.3 — FFmpeg Processing Consumer `[Catalog]`

Features: Consume TrackUploaded, run FFmpeg, update track status

Tables: `Catalog.TrackFiles` (Status → Ready or Failed)

- RabbitMQ consumer on queue `catalog.processing`
 - Read raw file path from message payload
 - Invoke FFmpeg via `System.Diagnostics.Process`
 - Full: `ffmpeg -i input.mp3 -codec:a aac -hls_time 10 -hls_list_size 0 output.m3u8`
 - Preview: same with `-t 30`
 - On success: `Status = Ready`, set `HlsPlaylistPath`, `PreviewPlaylistPath`, optionally clear `RawFilePath`
 - On failure: `Status = Failed`, set `FailureReason` from stderr
 - Emit `TrackReady` event (new `OutboxMessage`) on success — ES/Qdrant consumers subscribe to this later
 - Stuck track detector `BackgroundService`: `ProcessingStartedAt > 30 min` → mark Failed
 - Serilog logs FFmpeg exit code, duration, output path
-

2.4 — HLS Streaming Endpoints `[Streaming]`

Features: Serve HLS playlist, record play events

Tables: `Streaming.PlayHistory`

Redis: `play_count:{trackId}`, `playback_pos:{userId}:{trackId}`

- `GET /stream/{trackId}/playlist.m3u8` → 404 if Status != Ready, full playlist for auth users, preview for guests
 - Play event recorded on first segment request (debounced — not per segment)
 - `RecordPlayCommand` → insert `Streaming.PlayHistory` → write `OutboxMessage` (PlaybackRecorded) → Redis INCR `play_count:{trackId}`
 - Play count flush `BackgroundService` → every 5 min → flush Redis counters to `Catalog.Tracks.PlayCount`
 - Save/restore playback position via Redis `playback_pos:{userId}:{trackId}`
 - `GetResumePositionQuery` → return Redis position or 0
-

2.5 — Queue & Playback Session `[Streaming]`

Features: Queue management, shuffle, repeat, persist queue

Tables: `Streaming.QueueSessions`

Redis: `queue:{userId}`

- `UpdateQueueCommand` → write to Redis (fast) → sync to `QueueSessions` DB periodically
 - `GetQueueQuery` → read from Redis, fall back to DB on cache miss
 - Queue session includes: current track, position, shuffle flag, repeat mode, context
-

2.6 — Frontend: Player `[Frontend]`

Technologies: hls.js, React Context + useReducer

- Wire hls.js into player context
- Load `.m3u8` on play
- Play / pause / seek / volume controls
- Progress bar with scrubbing
- Skip next / previous (> 3s into track → restart; < 3s → go back)
- Shuffle and repeat mode buttons (Off / Repeat All / Repeat One)
- Now playing bar — persistent bottom bar across all pages
- Queue drawer UI
- Poll `GET /tracks/{id}/status` after upload to enable play button when Ready

Phase 3 — Social & Notifications

Goal: Following system, artist posts, notification bell, home feed, collaborative playlists.

New technologies: RabbitMQ fan-out consumers for notifications

Tables: All `Social.*` tables

3.1 — Following System `[Social]`

Features: Follow/unfollow artists and users

Tables: `Social.ArtistFollows`, `Social.UserFollows`

- `FollowArtistCommand` → insert `ArtistFollows` → increment `Catalog.Artists.FollowerCount` (cross-module, app enforces) → write `OutboxMessage` (`UserFollowedArtist`)
 - `UnfollowArtistCommand` → delete + decrement
 - `FollowUserCommand` / `UnfollowUserCommand` → update both follower/following counts on `Identity.Users`
 - `GetFollowersQuery` / `GetFollowingQuery` → paginated lists
 - xUnit tests
-

3.2 — Notifications `[Social]`

Features: In-app notification bell

Tables: `Social.Notifications`

- RabbitMQ consumer on `social.notifications` queue
 - Handles: `ArtistApproved`, `UserFollowedArtist`, `TrackReady`, `CollabInvite`
 - Inserts `Social.Notifications` row per event
 - `GetNotificationsQuery` → paginated, ordered by `CreatedAt DESC`
 - `MarkNotificationReadCommand` / `MarkAllReadCommand`
 - `GetUnreadCountQuery` → badge count for bell icon
 - xUnit tests
-

3.3 — Artist Posts [Social]

Features: Artist publishes update post, followers see it

Tables: Social.ArtistPosts

- CreateArtistPostCommand [Artist] → insert Social.ArtistPosts → write OutboxMessage (ArtistPublishedPost)
 - Consumer: batch insert Social.Notifications for all followers
 - GetArtistPostsQuery → paginated by artist
 - DeleteArtistPostCommand → soft delete
 - xUnit tests
-

3.4 — Home Feed [Social] [Frontend]

Features: Aggregated feed — new releases, artist posts, recently played

- GetHomeFeedQuery → join ArtistFollows + ArtistPosts + new Catalog.Albums from followed artists + last 5 from Streaming.PlayHistory
 - Frontend home page rendering feed cards
-

3.5 — Collaborative Playlists [Playlist]

Features: Owner invites collaborators, collaborators can edit

Tables: Playlist.PlaylistCollaborators

- InviteCollaboratorCommand → insert PlaylistCollaborators → write OutboxMessage (CollabInvite)
 - RemoveCollaboratorCommand
 - AddTrackToPlaylistCommand now checks ownership OR collaborator permission
 - xUnit tests
-

Phase 4 — Analytics

Goal: Artist dashboard, user personal stats, admin platform dashboard.

New technologies: .NET BackgroundService nightly Worker

Tables: All Analytics.* tables

4.1 — Analytics Rollup Worker [Analytics]

Features: Nightly aggregation from PlayHistory into all stats tables

- AnalyticsRollupWorker — BackgroundService runs nightly
 - Reads Streaming.PlayHistory for yesterday
 - Upserts Analytics.DailyTrackStats per track
 - Upserts Analytics.DailyArtistStats per artist
 - Upserts Analytics.MonthlyUserStats per user
 - Upserts Analytics.PlatformDailyStats for yesterday
 - Serilog logs duration + rows processed
-

4.2 — Artist Dashboard [Analytics] [Frontend]

Features: Stream chart, top tracks, unique listeners, follower trend

- GetArtistDashboardQuery → reads DailyTrackStats + DailyArtistStats
 - GetArtistTopTracksQuery → DailyTrackStats grouped by TrackId, date range param
 - GetFollowerTrendQuery → FollowerGained - FollowerLost per day from DailyArtistStats
 - Frontend: line charts, top tracks list
-

4.3 — User Listening Stats [Analytics] [Frontend]

Features: Top tracks, top artists, total listen time, Wrapped equivalent

- GetUserMonthlyStatsQuery → reads MonthlyUserStats for current month
 - GetUserWrappedQuery → aggregate all 12 MonthlyUserStats rows for a given year
 - Frontend: stats page, yearly Wrapped reveal
-

4.4 — Admin Dashboard [Analytics] [Infra] [Frontend]

Features: DAU chart, top charts, upload health, audit log viewer, user/artist management

- `GetPlatformStatsQuery` `[Admin]` → reads `PlatformDailyStats`
 - `GetTopChartsQuery` → `Catalog.Tracks` ordered by `PlayCount DESC`
 - `GetAuditLogsQuery` `[Admin]` → paginated `Infra.AuditLogs`
 - `GetAllUsersQuery` / `GetAllArtistsQuery` `[Admin]` → paginated with filters
 - `LockUserCommand` / `UnlockUserCommand` `[Admin]` → update + bulk revoke refresh tokens + write `AuditLogs`
 - Frontend: admin panel pages
-

Phase 5 — Search & Discovery

Goal: Full-text search via ElasticSearch, semantic search and recommendations via Qdrant.
New technologies: ElasticSearch 8 (`Elastic.Clients.Elasticsearch`), Qdrant (.NET SDK)
Add to Docker Compose: ElasticSearch + Qdrant containers

5.1 — ElasticSearch Setup & Indexing `[Catalog]`

Features: Index tracks when `TrackReady` fires, index artists when `ArtistApproved` fires

- RabbitMQ consumer on `catalog.search` queue (subscribes to `TrackReady` routing key)
 - Index document: `trackId`, `title`, `artistNames`, `albumTitle`, `genres`, `moods`, `duration`, `isExplicit`, `releaseYear`
 - Consumer on `catalog.artist.index` (subscribes to `ArtistApproved`)
 - Re-index on track metadata edit
 - Remove from index on soft delete
-

5.2 — Search Endpoints `[Catalog]` `[Frontend]`

Features: Full-text search, filters, autocomplete, recent searches

Tables: `Streaming.SearchHistory`

Redis: `autocomplete:{prefix}` TTL 5 min

- `SearchQuery` → ES query with filters (genre, mood, year range, explicit, duration)
- `AutocompleteQuery` → ES prefix query, Redis cached per prefix
- `SaveSearchHistoryCommand` → insert `SearchHistory` (delete oldest if > 50 rows)
- `GetRecentSearchesQuery` → last 10 from `SearchHistory`

- Frontend: search bar with autocomplete dropdown, results page with filter panel
-

5.3 — Qdrant Setup & Embeddings [Catalog]

Features: Store track vectors for semantic search and recommendations

- RabbitMQ consumer on `catalog.embeddings` queue (subscribes to `TrackReady`)
 - Generate embedding vector from genre/mood tags + metadata
 - Upsert into Qdrant collection `tracks` with payload `{ trackId, artistId, genreIds }`
 - Feed `Streaming.PlayHistory` signals (liked = positive, skipped = negative) into user vectors periodically via Worker
-

5.4 — Recommendation Endpoints [Catalog] [Frontend]

Features: Sounds like this, Recommended for you, Because you liked X

- `GetSimilarTracksQuery` → Qdrant nearest neighbours on track vector
 - `GetRecommendedForUserQuery` → Qdrant nearest neighbours on user behaviour vector
 - `GetBecauseYouLikedQuery` → given a liked track, find similar via Qdrant
 - Frontend: recommendation rows on home page and track pages
-

Phase 6 — Polish & Production Readiness

| **Goal:** Everything works end-to-end, tested, secure, ready to deploy.

6.1 — Testing Pass

- Integration tests with Testcontainers (real MSSQL container)
- 80%+ coverage on all Application layer handlers
- FluentValidation tests for every command validator
- RabbitMQ consumer tests (mock channel, test message handling logic)

6.2 — Security Hardening

- MIME type + magic byte validation on file uploads
- Stricter rate limiting on auth endpoints

- HTTPS enforced
- CORS locked to frontend origin only
- No stack traces in production error responses

6.3 — Observability

- Correlation IDs propagated to all RabbitMQ message headers
- Serilog enrichers: `UserId`, `TrackId`, `CorrelationId` on every relevant log
- Health checks cover: MSSQL, Redis, RabbitMQ, ES, Qdrant
- Slow query logging in EF Core (log queries > 500ms)

6.4 — Frontend Polish

- Responsive layout
- Loading skeletons on all async data
- Error boundaries
- Empty states
- Toast notifications for async actions (upload submitted, track processing complete, etc.)

6.5 — File Storage Swap (Optional)

- Implement `AzureBlobStorage : IFileStorage` or `S3FileStorage : IFileStorage`
- Register in DI instead of `LocalFileStorage` — zero other changes needed
- Pre-signed URL generation for HLS serving via CDN

Technology Decision Summary

Concern	Technology	Notes
Backend framework	.NET 8 Web API	
Frontend	React + Vite	
Primary database	MSSQL	7 schemas: Identity, Catalog, Streaming, Playlist, Social, Analytics, Infra

Concern	Technology	Notes
ORM	EF Core 8	One DbContext per module, Fluent API only, no data annotations
ID strategy	<code>Guid.CreateVersion7()</code>	Sequential GUIDs — no fragmentation, generated in app before DB insert
Cross-schema FKs	None	Plain <code>Guid</code> properties, application enforces consistency
CQRS	MediatR	Commands mutate state, queries use <code>AsNoTracking()</code> and return DTOs
Validation	FluentValidation	Pipeline behaviour — runs before every command handler automatically
Error handling	<code>Result<T></code> pattern	No business exceptions — errors returned as values
Auth	Custom JWT	15 min access token + 7 day refresh token, BCrypt passwords
Caching / sessions	Redis (StackExchange.Redis)	Play counts, queue, positions, token blacklist, rate limiting
Event bus	RabbitMQ raw client	No MassTransit — <code>RabbitMQ.Client</code> directly, <code>IEventBus</code> abstraction in Domain
Outbox	<code>Infra.OutboxMessages</code> table	Written in same EF transaction as business data, Worker publishes
Audio processing	FFmpeg CLI	<code>System.Diagnostics.Process</code> , outputs HLS segments
Audio streaming	HLS (.m3u8 + .ts)	hls.js on frontend, static files middleware in dev
File storage (dev)	Local disk	<code>wwwroot/uploads/</code> — <code>IFileStorage</code> abstraction, swap to Azure/S3 for prod
File storage (prod)	Azure Blob or AWS S3	Just swap the <code>IFileStorage</code> implementation
Full-text search	ElasticSearch 8	Phase 5 — never use MSSQL LIKE queries for search

Concern	Technology	Notes
Semantic search	Qdrant	Phase 5 — track embeddings + user behaviour vectors for recommendations
Logging	Serilog + Seq	Structured JSON, correlation IDs, Seq UI for dev debugging
Testing	xUnit + Moq	Unit tests per handler, Testcontainers for integration tests
Background jobs	.NET BackgroundService	OutboxProcessor, play count flush, analytics rollup, stuck track detector
Soft delete	EF Core global filter	<code>HasQueryFilter(e => !e.IsDeleted)</code> on all entities
Admin audit trail	<code>Infra.AuditLogs</code> table	Admin actions only — completely separate from Serilog runtime logs
Architecture	Modular Monolith + Vertical Slicing	One project per module, one folder per feature/use case

Quick Reference — What Goes Where

You want to...

You touch...

Add a new feature
Handler + Validator + Tests

→ New folder in the module: Command +

Add a new DB table
IEntityTypeConfiguration<T> + EF migration

→ New entity +

Publish a RabbitMQ event
transaction as business data

→ Write OutboxMessage in same EF

Consume a RabbitMQ event
the relevant module

→ New consumer class in Infrastructure of

Add a background job

→ New BackgroundService in Infrastructure

Do a Redis operation
Infrastructure layer

→ StackExchange.Redis directly in

Add an admin-only endpoint
Infra.AuditLogs

→ [Authorize(Roles = "Admin")] + write to

Log something
never string interpolation

→ ILogger<T> with named properties -

Search something abstraction	→ Elasticsearch query via <code>ISearchService</code>
Recommend something abstraction	→ Qdrant query via <code>IRecommendationService</code>
Read data for a UI screen returning a flat DTO	→ Query handler with <code>AsNoTracking()</code>
Mutate data SaveChangesAsync()	→ Command handler with full EF tracking +